

Bugboard - System/Software Design Document

Francesco Piscopo, Raffaele Ruggiero

19 marzo 2026

1 Indice

Indice

1	Indice	1
2	Criteri di design e architettura proposta	1
3	Scelta tecnologiche adottate	1
4	Persistenza dati	2
5	Uso versioning	2
6	Diagramma	2
7	Report qualità di codice	2
8	Testing	3

2 Criteri di design e architettura proposta

Abbiamo adottato una architettura **Client-Server**, in cui il server gestisce il database, scrittura e lettura su quest'ultimo e i sistemi di autenticazione.

Il motivo dietro a questa scelta è semplice, è una architettura **classica e funzionale** in cui i client eseguono le richieste ad un unico server, il quale esegue una serie di servizi. Sicuramente il fatto che ci sia un single point of failure potrebbe rappresentare un problema, ma considerando i problemi pratici che possono esserci dietro all'implementazione di un'altra architettura e soppesando pro e contro al riguardo siamo giunti a questa scelta.

Inoltre abbiamo adoperato anche una architettura **3tier**, con UI(front-end), logica sul server(backend) e gestione del database tramite postgresql.

3 Scelta tecnologiche adottate

Abbiamo optato per le seguenti tecnologie:

- **React** per il frontend, questo è dato dal fatto che il web development è un settore in continua crescita e realizzare web app è ottimale in quanto garantisce il facile funzionamento su quasi tutti i sistemi informatici utilizzati dai nostri utenti target, e React con la sua ampia documentazione e community facilita la realizzazione di queste applicazioni, seppur per progetti piccoli come il nostro abbia introdotto anche una serie di problemi di apprendimento in quanto c'era poca familiarità con quest'ultimo.
- **Next.JS e Typescript** per il backend, questo perché sono rispettivamente un framework e un linguaggio di programmazione adatti proprio a funzionare nel full-stack web development

con React e quindi risultava una scelta molto semplice. Nonostante ciò rimane sicuramente problematico il problema dell'apprendimento di nuove tecnologie da pochissime basi.

- **Docker** per la virtualizzazione del sistema che risulta sicuramente la scelta più semplice per adempiere a questo ruolo in quanto indiscussa tecnologia in questo campo dell'informatica.
- **PostgreSQL** come DBMS, questo è perché è sicuramente il sistema SQL con il quale siamo più competenti ed è anche il sistema SQL più aperto e funzionale rispetto ad altri sul mercato (comparato ad altri tipo quelli di Oracle).

4 Persistenza dati

Questa avviene tramite l'utilizzo di un DBMS, le qualità di quest'ultimo sono varie ma le più importanti sono la **sicurezza** e, appunto, la **persistenza** delle informazioni riguardanti gli utenti e gli issues, mentre la persistenza dell'accesso viene gestita tramite cookie sul filesystem del client.

5 Uso versioning

Per il versioning è stato utilizzato github, sia con funzionalità di branching (non intenzionale) <https://github.com/urubooru/BugBoard26/tree/workingBranchHopefully>

6 Diagramma

Il seguente diagramma non comprende una serie di librerie importate da fuori.

Tra l'altro vale la pena notare che non è un vero e proprio diagramma di classe in quanto solo una classe esiste nell'effettivo nel sistema.

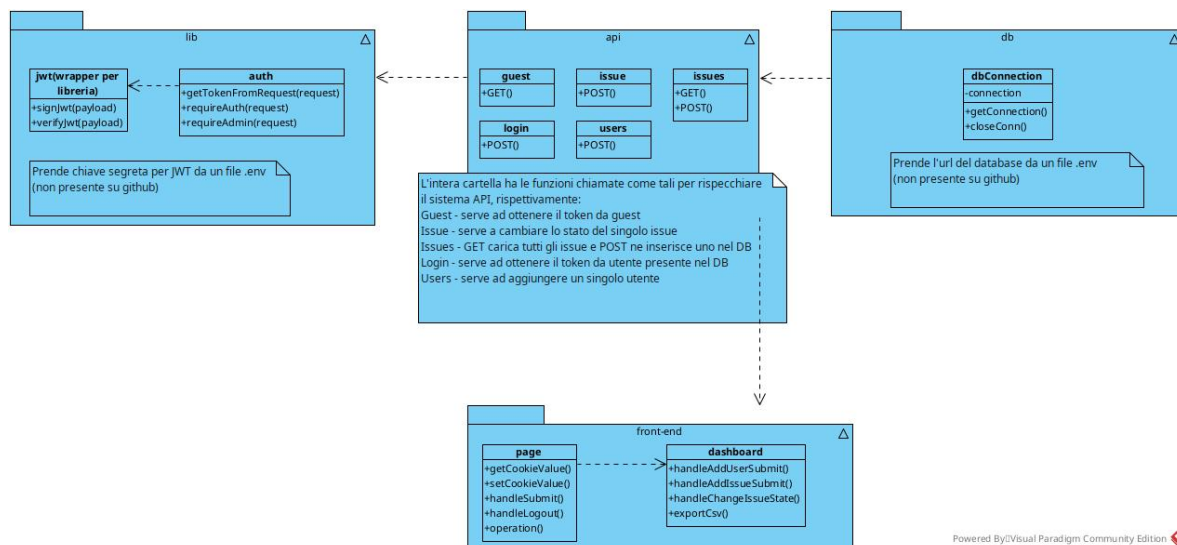


Figura 1: Diagramma delle funzioni.

7 Report qualità di codice

Durante la produzione dell'applicativo è stato utilizzato lo strumento di analisi statico SonarQube, il quale ha aiutato a realizzare codice di qualità e gli ultimi warning rimasti sono stati reputati nulla che necessitasse di bug-fixing.

8 Testing

L'unit testing è stato realizzato tramite l'utilizzo del framework **vitest**, questo è stato cominato a una leggera modifica di una delle 2 unità utilizzate durante la fase di development per creare del "mock code".

Oltre a questo è stato eseguito tanto testing manuale da parte dei membri del team che ha portato a scovare vari bug nella fase di scrittura del codice in se e quindi non più presenti.